

■ Library Management System

Project Requirements & Development Roadmap

Stack	ASP.NET Core MVC 8 + Entity Framework Core + SQL Server
Pattern	Repository Pattern + Unit of Work
Frontend	Razor Views + Bootstrap 5
Auth	ASP.NET Core Identity (Admin / Member roles)
Phases	9 development phases — from Setup to Authentication
Prepared for	Abdo — Training Project

This document covers all models, features, requirements, and development phases needed to build a complete Library Management System from scratch using ASP.NET Core MVC.

1. Project Overview

What are we building?

A full-featured Library Management System that allows librarians to manage books, authors, and members, while tracking borrow and return operations. The system supports search, filtering, pagination, and role-based access control.

Core Modules

Module	Description	Priority
Books	Add/edit/delete books, availability	High
Authors	Manage author profiles	High
Members	Register and manage library members	High
Borrow System	Issue & return books, due dates	High
Search	Search by title/author/genre	Medium
Pagination	Paginated lists for all entities	Medium
Admin Panel	Role-based dashboard for admins	Medium
Authentication	Login, register, roles	Bonus

2. Database Models (Entities)

Book

Type	Property	Notes
int	Id	Primary Key
string	Title	Required, max 200 chars
string	ISBN	Required, unique
string	Genre	e.g. Fiction, Science
int	PublishedYear	Year of publication
int	TotalCopies	Total copies in library
int	AvailableCopies	Updated on borrow/return
int	AuthorId	FK -> Author
Author	Author	Navigation property

Author

Type	Property	Notes
int	Id	Primary Key
string	FirstName	Required
string	LastName	Required
string	Bio	Optional biography
string	Nationality	Optional
ICollection	Books	Navigation property

Member

Type	Property	Notes
int	Id	Primary Key
string	FullName	Required
string	Email	Required, unique
string	Phone	Optional
string	Address	Optional
DateTime	MemberSince	Auto set on create
bool	IsActive	Active/suspended

ICollection**BorrowRecords**Navigation property

BorrowRecord

Type	Property	Notes
int	Id	Primary Key
int	BookId	FK -> Book
int	MemberId	FK -> Member
DateTime	BorrowDate	Auto set on borrow
DateTime	DueDate	BorrowDate + 14 days
DateTime?	ReturnDate	Null if not returned yet
bool	IsReturned	false by default
Book	Book	Navigation property
Member	Member	Navigation property

Entity Relationships

Author 1 ■■■■■■< Books (One Author has many Books)

Book 1 ■■■■■■< BorrowRecords (One Book can be borrowed many times)

Member 1 ■■■■■■< BorrowRecords (One Member can borrow many books)

3. Functional Requirements

Books Management

- 1 View all books in a paginated, searchable list
- 2 Add a new book with title, ISBN, genre, year, copies, and author
- 3 Edit book details
- 4 Delete a book (only if no active borrows exist)
- 5 View book details including available copies
- 6 Filter books by genre or author

Authors Management

- 1 View all authors with their book count
- 2 Add a new author with name, bio, and nationality
- 3 Edit author details
- 4 Delete an author (only if they have no books)
- 5 View all books by a specific author

Members Management

- 1 View all members in a paginated list
- 2 Register a new member with name, email, and phone
- 3 Edit member information
- 4 Deactivate/activate a member account
- 5 View member borrow history

Borrow System

- 1 Issue a book to a member (BorrowDate = today, DueDate = today + 14 days)
- 2 Check book availability before issuing
- 3 Return a book (set ReturnDate = today, IsReturned = true)
- 4 Automatically decrease AvailableCopies on borrow
- 5 Automatically increase AvailableCopies on return
- 6 View all active borrows (not returned yet)

-
- 7 View overdue borrows (DueDate < today and not returned)
-

Search & Filter

- 1 Search books by title or ISBN

- 2 Filter books by genre

- 3 Filter books by author

- 4 Search members by name or email

- 5 Paginate all result sets (10 items per page)
-

4. Development Roadmap — 9 Phases

1

Project Setup & Database Design

Create solution, configure EF Core, connect to SQL Server

- Create ASP.NET Core MVC project in Visual Studio
- Install packages: Microsoft.EntityFrameworkCore.SqlServer, Microsoft.EntityFrameworkCore.Tools
- Create appsettings.json connection string
- Register DbContext in Program.cs
- Plan folder structure: Models/, Controllers/, Views/, Repositories/, ViewModels/

2

Models & EF Core

Define entities, DbContext, Migrations, and Seed data

- Create Book.cs, Author.cs, Member.cs, BorrowRecord.cs in Models/
- Add Data Annotations: [Required], [MaxLength], [EmailAddress]
- Create LibraryDbContext with DbSet for each entity
- Configure relationships using Fluent API or navigation properties
- Run: Add-Migration InitialCreate then Update-Database
- Add seed data via HasData() in OnModelCreating

3

Repository Pattern

Abstract data access with IRepository and UnitOfWork

- Create IRepository generic interface with GetAll, GetById, Add, Update, Delete
- Create Repository : IRepository base implementation
- Create specific interfaces: IBookRepository, IAuthorRepository, IMemberRepository, IBorrowRepository
- Implement each with custom queries (e.g. GetAvailableBooks)
- Create IUnitOfWork and UnitOfWork to wrap all repositories
- Register all in Program.cs as Scoped services

4

Controllers & Actions

Build CRUD controllers using repository via UnitOfWork

- Create BooksController with Index, Details, Create (GET/POST), Edit (GET/POST), Delete (GET/POST)
- Create AuthorsController with same CRUD structure
- Create MembersController with same CRUD structure
- Create BorrowController with Issue, Return, ActiveBorrows, Overdue actions
- Use IUnitOfWork via constructor injection in each controller
- Return appropriate ActionResult types (View, RedirectToAction, NotFound)

5

Razor Views

Build all UI pages using
Razor syntax and Layout

- Create `_Layout.cshtml` with navbar, footer, Bootstrap 5 CDN
- Create `Index.cshtml` for each entity with HTML table and action buttons
- Create `Create.cshtml` and `Edit.cshtml` using `asp-for` tag helpers
- Create `Details.cshtml` and `Delete.cshtml` (confirmation page)
- Create `_BorrowRow.cshtml` partial view for reuse
- Use `ViewBag/ViewData` for page titles and dropdown lists

6

ViewModels & Validation

Add `ViewModels` and
client+server side validation

- Create `BookViewModel`, `MemberViewModel`, `BorrowViewModel` in `ViewModels/`
- Add `[Required]`, `[StringLength]`, `[Range]` annotations
- Show validation messages in Views with `asp-validation-for`
- Check `ModelState.IsValid` in POST actions before saving
- Add jQuery Unobtrusive Validation scripts to `_Layout`

7

Borrow System

Implement core library
feature: book issuing and
returns

- `BorrowController.Issue(GET)`: show form with available books and active members
- `BorrowController.Issue(POST)`: validate availability, create `BorrowRecord`, decrement `AvailableCopies`
- `BorrowController.Return(POST)`: set `ReturnDate`, `IsReturned=true`, increment `AvailableCopies`
- `BorrowController.ActiveBorrows`: list all unreturned records
- `BorrowController.Overdue`: list records where `DueDate < DateTime.Today` and not returned
- Show borrow history per member in `MembersController.Details`

8

Search, Filter & Pagination

Add search and pagination
to all list pages

- Install `X.PagedList.Mvc.Core` NuGet package
- Add search parameter to Index actions: `string? searchTerm`, `string? genre`
- Use `LINQ Where()` to filter results before paging
- Return `IPagedList` to views
- Add search form and pagination controls to Index views
- Show 'X results found' count and current page info

9

Authentication (Bonus)

Add login, registration, and
role-based access

- Install Microsoft.AspNetCore.Identity.EntityFrameworkCore
- Create LibraryUser : IdentityUser and update DbContext
- Run new migration for Identity tables
- Create AccountController with Register and Login actions
- Seed Admin role and default admin user on startup
- Add [Authorize] and [Authorize(Roles='Admin')] to protected controllers
- Show/hide nav items based on logged-in user role

5. Non-Functional Requirements

Architecture	N-tier: Presentation (Views) → Application (Controllers) → Data (Repositories)
ORM	Entity Framework Core — Code First approach with Migrations
Validation	Server-side (Data Annotations + ModelState) and client-side (jQuery Unobtrusive)
UI Framework	Bootstrap 5 via CDN — responsive, mobile-friendly
Error Handling	Custom 404 and 500 pages, try-catch in critical operations
Security	CSRF protection via AntiForgeryToken on all POST forms
Code Quality	DRY principle, meaningful naming, XML comments on public methods
DB Indexing	Index on Book.ISBN, Member.Email for fast lookups

6. Folder Structure

```

LibraryMVC/
├── Controllers/
│   ├── BooksController.cs
│   ├── AuthorsController.cs
│   ├── MembersController.cs
│   └── BorrowController.cs
├── Models/
│   ├── Book.cs
│   ├── Author.cs
│   ├── Member.cs
│   └── BorrowRecord.cs
├── ViewModels/
│   ├── BookViewModel.cs
│   ├── MemberViewModel.cs
│   └── BorrowViewModel.cs
├── Repositories/
│   ├── IRepository.cs
│   ├── Repository.cs
│   ├── IBookRepository.cs
│   ├── BookRepository.cs
│   ├── IUnitOfWork.cs
│   └── UnitOfWork.cs
├── Data/
│   └── LibraryDbContext.cs
├── Views/
│   ├── Books/ (Index, Create, Edit, Details, Delete)
│   ├── Authors/ (Index, Create, Edit, Details, Delete)
│   └── Members/ (Index, Create, Edit, Details, Delete)

```

```
■ ■■■ Borrow/ (Issue, Return, ActiveBorrows, Overdue)
■ ■■■ Shared/ (_Layout, _ValidationScriptsPartial)
■■■ appsettings.json
```

7. NuGet Packages Required

Package	Version	Purpose
Microsoft.EntityFrameworkCore.SqlServer	8.x	EF Core SQL Server provider
Microsoft.EntityFrameworkCore.Tools	8.x	Migrations CLI (Add-Migration, Update-Database)
Microsoft.EntityFrameworkCore.Design	8.x	Design-time tools for EF Core
Microsoft.AspNetCore.Identity.EntityFrameworkCore	8.x	ASP.NET Core Identity (Bonus phase)
X.PagedList.Mvc.Core	latest	Pagination helper for MVC views
Microsoft.AspNetCore.Mvc.Razor.RuntimeCompilation	8.x	Live Razor view refresh in dev

Install all at once via Package Manager Console:

```
Install-Package Microsoft.EntityFrameworkCore.SqlServer
```

```
Install-Package Microsoft.EntityFrameworkCore.Tools
```

```
Install-Package Microsoft.EntityFrameworkCore.Design
```

```
Install-Package X.PagedList.Mvc.Core
```